PERFORMANCE ENGINEERING

Übungsblatt zum WJAX-Workshop

© Michael Inden 2026

Dieses Arbeitsblatt folgt der Struktur des Vortrags. Die Aufgaben verbinden Analyse, Implementierung und Performance-Messung. Zu ausgewählten Aufgaben sind Ausgangsimpementierungen enthalten, die du untersuchen und schrittweise verbessern kannst. Die Zeitangaben sind grobe Richtwerte für Java-Entwickler mit mittlerer Erfahrung – je nach Kenntnisstand wird mehr Zeit für Recherche und Vertiefungen benötigt.

🧠 Performance verstehen: Big-O und Skalierung

Aufgabe 1: Laufzeitkomplexitäten klassifizieren

Schwierigkeit: leicht · Bearbeitungszeit: ca. 10 Min.

Bearbeite das separate interaktive O-Notation-Quiz. Ordne jeder Implementierung eine Laufzeitkomplexität zu und begründe deine Einschätzung anhand der Anzahl ausgeführter Operationen.

- Achte auf Schleifen mit konstanter Grenze und auf verschachtelte Schleifen.
- Unterscheide lineares von logarithmischem Wachstum, z. B. wenn eine Laufvariable pro Schritt verdoppelt oder halbiert wird.

Aufgabe 2: Potenzfunktion effizient berechnen

Schwierigkeit: mittel · Bearbeitungszeit: ca. 15–20 Min.

Gegeben ist eine Implementierung, die x^n durch wiederholte Multiplikation `berechnet` (z. B. $4^2 = 16$).

```
long exponentiationBruteForce(long x, int n) {
    long result = 1;
    for (int i = 0; i < n; i++) {
        result *= x;
    }
    return result;
}
```

1. Analysiere die Laufzeitkomplexität in Abhängigkeit vom Exponenten n.
2. Überlege, wie sich der Exponent in jedem Schritt halbieren lässt.
3. Implementiere eine verbesserte Variante.
4. Vergleiche die Laufzeit beider Varianten.

Aufgabe 3: Erster Mini-Benchmark – Duplikate finden

Schwierigkeit: leicht bis mittel · Bearbeitungszeit: ca. 15–20 Min.

Vergleiche zwei Implementierungen derselben Aufgabe: Eine Variante prüft alle Elementpaare, die andere merkt sich bereits gesehene Werte in einem `HashSet`.

```
static boolean hasDuplicatesNaive(int[] values) {
    for (int i = 0; i < values.length; i++) {
        for (int j = i + 1; j < values.length; j++) {
            if (values[i] == values[j])
                return true;
        }
    }
    return false;
}

static boolean hasDuplicatesHashSet(int[] values) {
    Set<Integer> seen = new HashSet<>();
    for (int value : values) {
        if (!seen.add(value))
            return true;
    }
    return false;
}
```

Nachfolgend sind einige nützliche Code-Schnipsel für die Performance-Messung gezeigt:

```
import java.util.concurrent.TimeUnit;

private static boolean result;

static long measureNanos(Runnable task, int warmups, int repetitions) {
    for (int i = 0; i < warmups; i++)
        task.run(); // einfaches Warm-up

    long start = System.nanoTime();
    for (int i = 0; i < repetitions; i++)
        task.run();

    return (System.nanoTime() - start) / repetitions;
}

int repetitions = 5;
long naiveTime = measureNanos(() -> result = hasDuplicatesNaive(values), repetitions);
long setTime = measureNanos(() -> result = hasDuplicatesHashSet(values), repetitions);

long naiveMicros = TimeUnit.NANOSECONDS.toMicros(naiveTime);
long setMicros   = TimeUnit.NANOSECONDS.toMicros(setTime);
double speedup = setTime > 0 ? (double) naiveTime / setTime : Double.NaN;

System.out.printf(
    "n=%d | Naiv: %d µs | HashSet: %d µs | Speedup: %.2fx%n",
    values.length, naiveMicros, setMicros, speedup
);
```

- Teste mehrere Eingabegrößen, etwa 1.000, 10.000 und 100.000. und gib Laufzeiten und Speedup aus: `Time`(naiv) / `Time`(`HashSet`).
- Verwende für den Worst Case ein Array ohne Duplikate, damit beide Varianten alle Elemente prüfen.
- Wiederhole die Messungen und dokumentiere, warum ein einzelner Lauf kein belastbarer `Benchmark` ist.

Messhinweis Die Methode `System.currentTimeMillis()` liefert die aktuelle Zeit in Millisekunden; für verstrichene Laufzeit ist die Methode `System.nanoTime()` geeigneter. Bei der Umrechnung ist `TimeUnit.NANOSECONDS.toMillis(nanos)` nützlich, rechnet in ganze Millisekunden um und schneidet Nachkommastellen ab. Für kurze Läufe ist `toMicros()` oft aussagekräftiger. Beachte unbedingt dass Warm-up, JIT, Garbage `Collection` und andere Prozesse einfache Messungen beeinflussen. Eine solche Messung erlaubt eine erste Einordnung, ist aber kein belastbarer `Benchmark`; dafür später `VisualVM` und optional `JMH` verwenden.

Aufgabe 4: Utility-Klasse `DemoDataCreator` erweitern

Schwierigkeit: leicht bis mittel · Bearbeitungszeit: ca. 30 Min.

Für die Entwicklung und das Testen von Algorithmen werden häufig künstliche Testdaten benötigt. Anstatt Arrays, Listen und Strings immer manuell zu erstellen, soll eine Hilfsklasse `DemoDataCreator` diese Daten automatisch generieren können.

Eine Klasse zur Erzeugung und zum Mischen von `int[]` ist bereits teilweise implementiert. Sie enthält Methoden für aufsteigend und absteigend sortierte Arrays sowie zufällig gemischte mithilfe eines Fisher-Yates-Shuffle.

Teil A – `Integer`-Listen erzeugen

Erweitere die Klasse um folgende Methoden:

```
public static List<Integer> generateAscSortedSizedIntList(int size)
public static List<Integer> generateDescSortedSizedIntList(int size)
public static List<Integer> generateRandomlyPopulatedIntList(int size)
```

- Erzeuge aufsteigend sortierte Listen mit den Werten von 0 bis size - 1.
- Erzeuge absteigend sortierte Listen mit den Werten von size - 1 bis 0.
- Erzeuge zufällig angeordnete Listen, die dieselben Werte enthalten.

Teil B – Zufällige Strings erzeugen

Implementiere eine Methode, die einen zufälligen String mit vorgegebener Länge erzeugt:

```
public static String generateRandomString(int length)
```

Verwende standardmässig folgenden Zeichenvorrat:

```
ABCDEFGHIJKLMNOPQRSTUVWXYZ
abcdefghijklmnopqrstuvwxyz
0123456789
```

Ergänze ausserdem eine überladene Methode für einen eigenen Zeichenvorrat:

```
public static String generateRandomString(int length, String characters)
```

Beispiele:

```
generateRandomString(10); // z. B. aB8xP2kLm9
generateRandomString(8, "ABC"); // z. B. ABACCBAB
```

📦 Wichtige Datenstrukturen und ihre Eigenschaften

Aufgabe 1: Datenstruktur passend zur Operation wählen

Schwierigkeit: leicht · Bearbeitungszeit: ca. 10–15 Min.

Wähle für jede Anforderung eine geeignete Java-Datenstruktur und begründe deine Entscheidung anhand der erwarteten Operationen und Laufzeiten.

Anforderung	Deine Wahl / Begründung
Häufiger Zugriff über einen Index	
Häufiges Einfügen und Entfernen an beiden Enden	
Schnelle Prüfung, ob ein Wert bereits vorkommt	
Zuordnung einer ID zu einem Objekt	
Sortierte Iteration über Schlüssel	

Aufgabe 2: Wort-Häufigkeiten zählen

Schwierigkeit: mittel · Bearbeitungszeit: ca. 15–20 Min.

Gegeben ist eine Liste von Wörtern. Bestimme, wie oft jedes Wort unabhängig von Groß- und Kleinschreibung vorkommt. Die folgende Ausgangsimplementierung durchsucht die bisher ermittelte Ergebnisliste bei jedem neuen Wort.

```
record WordCount(String word, int count) {}

static List<WordCount> countNaive(List<String> words) {
    List<WordCount> result = new ArrayList<>();
    for (String word : words) {
        int index = -1;
        for (int i = 0; i < result.size(); i++) {
            if (result.get(i).word().equalsIgnoreCase(word)) {
                index = i;
                break;
            }
        }
        if (index >= 0) {
            WordCount old = result.get(index);
            result.set(index, new WordCount(word.toLowerCase(), old.count() + 1));
        } else {
            result.add(new WordCount(word, 1));
        }
    }
    return result;
}
```

- Analysiere den Aufwand der obigen Implementierung einer verschachtelten Suche.
- Realisiere dieselbe Funktionalität mithilfe einer `HashMap<String, Integer>`.
- Vergleiche Laufzeit und Speicherbedarf. Welche Variante ist bei vielen unterschiedlichen Wörtern günstiger?

Aufgabe 3: Queue optimieren – `ArrayList`, `LinkedList` und `ArrayDeque`

Schwierigkeit: mittel bis anspruchsvoll · Bearbeitungszeit: ca. 30 Min.

Betrachte die folgende Queue, die zur Datenhaltung intern eine `ArrayList` verwendet. Das Einfügen erfolgt am Ende; beim Entfernen wird das erste Element gelöscht.

```
class ArrayListBasedQueue<T> {
    private final List<T> values = new ArrayList<>();

    public void enqueue(T value) {
        values.addLast(value);
    }

    public T dequeue() {
        if (values.isEmpty())
            return null;
        return values.removeFirst();
    }

    public boolean isEmpty() {
        return values.isEmpty();
    }
}
```

Teil A – Analyse

1. Warum ist die Laufzeit der Methode `enqueue()` bei Verwendung einer `ArrayList` amortisiert O(1), obwohl das interne Array gelegentlich vergrößert werden muss?
2. Welche Laufzeit besitzt `dequeue()`? Begründe deine Antwort anhand der internen Datenstruktur.
3. Welche Gesamtkomplexität entsteht, wenn n Elemente nacheinander entnommen werden?

Teil B – Optimierung 1: `LinkedList` verwenden

Ersetze die `ArrayList` durch eine `LinkedList`. Verwende das Deque-Interface und füge am Ende ein bzw. entnimm am Anfang. Wie verändert sich die Laufzeit für beide Operationen?

```
class LinkedListBasedQueue<T> {
    private final Deque<T> values = new LinkedList<>();

    public void enqueue(T value) {
        values.addLast(value);
    }

    public T dequeue() {
        if (values.isEmpty())
            return null;
        return values.removeFirst();
    }

    public boolean isEmpty() {
        return values.isEmpty();
    }
}
```

Teil C – Optimierung 2: Built-in-Klasse `ArrayDeque`

1. Ersetze die Datenhaltung durch die im JDK-enthaltene Klasse `ArrayDeque`.
2. Vergleiche die Laufzeiten der Kombination der Aktionen Einfügen und Entnehmen in allen Varianten.

🔎 Suchen und Sortieren

Aufgabe 1: Insertion Sort und Merge Sort vergleichen

Schwierigkeit: mittel · Bearbeitungszeit: ca. 30 Min.

Miss die Laufzeiten von Insertion Sort und Merge Sort für verschiedene Eingabegrößen. Verwende identische Daten für beide Algorithmen und prüfe unterschiedliche Eingabeverteilungen: zufällig, bereits sortiert und umgekehrt sortiert. Verwende als konkrete Eingabe eine `ArrayList`, da beide Implementierungen indexbasiert arbeiten.

```
static List<Integer> insertionSort(List<Integer> values) {

    List<Integer> result = new ArrayList<>();
    for (int current : values) {
        int i = 0;

        // Richtige Position suchen
        while (i < result.size() && result.get(i) <= current) {
            i++;
        }

        // Element an der richtigen Position einfügen
        result.add(i, current);
    }

    return result;
}
```

Merge Sort verwendet einen Teile-und-Herrsche-Ansatz und kann rekursiv wie folgt implementiert werden:

```
static List<Integer> mergeSort(List<Integer> values) {

    if (values.size() <= 1) {
        return values;
    }

    int mid = values.size() / 2;

    List<Integer> left = new ArrayList<>(values.subList(0, mid));
    List<Integer> right = new ArrayList<>(values.subList(mid, values.size()));

    left = mergeSort(left);
    right = mergeSort(right);

    List<Integer> result = merge(left, right);

    return result;
}
```

- Ordne den Algorithmen jeweils eine passende Zeitkomplexität zu.
- Führe einige Benchmark-Messungen etwa mit 1.000, 10.000 und 100.000 Elementen durch.
- Was beobachtest du für kleine oder nahezu sortierte Eingabedatenbestände?

Aufgabe 2: Hybrides Sortieren

Schwierigkeit: mittel · Bearbeitungszeit: ca. 30 Min.

Kombiniere Merge Sort und Insertion Sort: Verwende Merge Sort für große Teilbereiche und Insertion Sort, sobald ein Teilbereich eine kleine, von dir gewählte Größe unterschreitet.

- Implementiere den Wechsel über einen Schwellwert k.
- Teste mehrere Werte für k und verschiedene Eingabeverteilungen.
- Vergleiche das Ergebnis mit reinem Merge Sort. Begründe, warum ein hybrider Ansatz sinnvoll sein kann.

Aufgabe 3: Insertion Sort auf Statement-Ebene optimieren

Schwierigkeit: einfach · Bearbeitungszeit: ca. 15 Min.

Die folgende Ausgangsimplementierung bewegt das einzusortierende Element durch wiederholte Vertauschungen nach links. Untersuche die Anzahl der Lese- und Schreibzugriffe.

```
static int moveToInsertionPos(List<Integer> values, int i) {
    int pos = i;
    while (pos > 0 && values.get(pos - 1) > values.get(pos)) {
        int tmp = values.get(pos - 1);
        values.set(pos - 1, values.get(pos));
        values.set(pos, tmp);
        pos--;
    }
    return pos;
}

static void insertionSort(List<Integer> values) {
    for (int i = 1; i < values.size(); i++) {
        moveToInsertionPos(values, i);
    }
}
```

Optimiere die innere Schleife so, dass das einzusortierende Element zwischengespeichert und nicht bei jedem Schritt vertauscht wird. Vergleiche die Anzahl der Schreibzugriffe und die Lesbarkeit deiner Lösung.

Welche Vorteile bezüglich Laufzeit ergeben sich? Ändert sich etwas an der Komplexitätsklasse? Wie sinnvoll ist diese Optimierung?

💡 Cleverere Lösungen: Two Pointers, HashMaps und Sortierung

Aufgabe 1a: Two Sum auf sortierten Daten

Schwierigkeit: mittel · Bearbeitungszeit: ca. 30 Min.

Gegeben ist ein aufsteigend sortiertes Array mit unterschiedlichen Ganzzahlen, etwa `[1, 2, 4, 7, 10, 11, 17]`. Finde heraus, ob zwei Werte die Zielsumme k ergeben, etwa den Wert 15. Optimiere die folgende Brute-Force-Implementierung durch Einsatz der Two-Pointer-Technik.

```
static boolean hasPairWithSumBruteForce(int[] values, int k) {
    for (int i = 0; i < values.length; i++) {
        for (int j = i + 1; j < values.length; j++) {
            if (values[i] + values[j] == k) {
                return true;
            }
        }
    }
    return false;
}
```

- Vollziehe zuerst die Brute-Force-Lösung nach, probiere sie aus und bestimme ihre Komplexität.
- Für Two-Pointer: Nutze und platzieren einen linken Zeiger auf den Anfang und einen rechten Zeiger auf das Ende.
- Bewege abhängig von der Summe jeweils einen der beiden Zeiger. Begründe, warum das korrekt ist.
- Vergleiche die Laufzeit beider Lösungen.
- Beachte Randfälle wie `[1, 2, 5, 6, 7]` und eine Suche nach dem Wert 4. Wieso ist das speziell? Was könnte da ggf. schiefgehen? Was ändert sich für diesen Datenbestand `[1, 2, 3, 5, 6, 7]`.

Aufgabe 1b: Two Sum auf unsortierten Daten

Schwierigkeit: mittel · Bearbeitungszeit: ca. 25 Min.

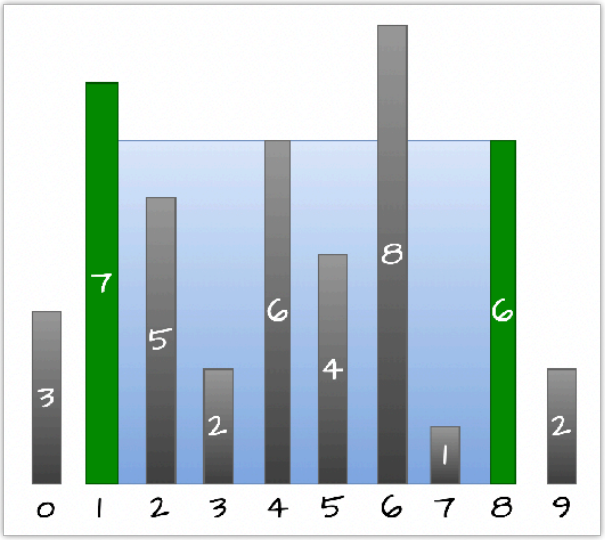
Gegeben ist ein unsortiertes Array mit unterschiedlichen Ganzzahlen, beispielsweise `[1, 2, 17, 11, 7, 4, 10]`. Finde heraus, ob zwei unterschiedliche Werte die Zielsumme k ergeben, etwa den Wert 15. Optimiere die obige folgende Brute-Force-Implementierung. Überlege dir, welche Datenstruktur oder welcher Algorithmus helfen könnte, die Suche effizienter zu gestalten.

- Überlege, weshalb die Two-Pointer-Technik aus Aufgabe 1a hier nicht ohne Weiteres funktioniert.
- Entwickle eine effizientere Lösung für unsortierte Daten. Welche Rolle könnte dabei ein `HashSet` oder eine `HashMap` spielen?
- Beachte Randfälle wie `[1, 2, 6, 7, 5]` und eine Suche nach dem Wert 4. Wieso ist das speziell? Was könnte dabei schiefgehen? Was ändert sich für diesen Datenbestand `[1, 2, 3, 5, 6, 7]`?
- Vergleiche Laufzeit und Speicherbedarf deiner Lösung mit der Brute-Force-Implementierung.

Aufgabe 2: Maximale Fläche zwischen zwei Linien

Schwierigkeit: mittel · Bearbeitungszeit: ca. 20–25 Min.

Gegeben ist eine Liste mit Höhenangaben von Stäben. Zwei Positionen i und j bilden einen Container bzw. ein Rechteck. Seine Fläche beträgt (j − i) × `min`(height[i], height[j]). Bestimme die maximal mögliche Fläche für eine gegebene Folge von Höhenangaben.



Beispielhafte Höhenwerte; zwei grüne Begrenzungen spannen einen möglichen Container auf.

Gegeben ist diese Brute-Force-Implementierung:

```
record PosHeightArea(int left, int right, int height, long area) {}

static PosHeightArea largestAreaBruteForce(List<Integer> values) {
    PosHeightArea best = new PosHeightArea(0, 0, 0, 0);
    for (int i = 0; i < values.size(); i++) {
        for (int j = i + 1; j < values.size(); j++) {
            int width = j - i;
            int minHeight = Math.min(values.get(i), values.get(j));
            long area = (long) width * minHeight;
            if (area > best.area()) {
                best = new PosHeightArea(i, j, minHeight, area);
            }
        }
    }
    return best;
}
```

- Analysiere die obige Brute-Force-Lösung und ihre Laufzeitkomplexität.
- Entwickle eine Two-Pointer-Lösung und begründe, warum jeweils die Seite mit der kleineren Höhe verschoben wird. Wie ändert sich die Laufzeitkomplexität?
- Überprüfe deine Implementierung mit den Werten aus der Abbildung. Es wird 7 * 6 = 42 als Ergebnis erwartet.
- Führe Performance-Vergleiche durch, um die Aussagen zur Laufzeitkomplexität zu bestätigen.

Aufgabe 3: Anagramme erkennen

Schwierigkeit: leicht / mittel · Bearbeitungszeit: ca. 20–25 Min.

Zwei Wörter sind Anagramme, wenn sie dieselben Zeichen mit derselben Häufigkeit enthalten. Das gilt etwa für „Ampel“, „Lampe“ und „Palme“. Groß-/Kleinschreibung soll bei der Überprüfung ignoriert werden; Leerzeichen und Sonderzeichen können zunächst ausgeschlossen werden.

```
static boolean isAnagramBruteForce(String word1, String word2) {
    if (word1.length() != word2.length())
        return false;

    word1 = word1.toLowerCase();
    word2 = word2.toLowerCase();
    char[] remaining = word2.toCharArray();

    for (int i = 0; i < word1.length(); i++) {
        boolean found = false;
        for (int j = 0; j < remaining.length; j++) {
            if (word1.charAt(i) == remaining[j]) {
                found = true;
                remaining[j] = '-'; // bereits verwendet
                break;
            }
        }
        if (!found)
            return false;
    }
    return true;
}
```

- Analysiere die Laufzeit der Ausgangsimplementierung.
- Implementiere eine Variante durch Sortieren. Welche Komplexität erreichen wir dann?
- Implementiere alternativ eine Häufigkeitszählung mit einem Array oder einer `Map`. Verbessert sich dadurch die Komplexitätsklasse? Wenn ja, was ist der Trade-off?

Aufgabe 4: Minimale Distanz

Schwierigkeit: mittel · Bearbeitungszeit: ca. 25–30 Min.

Gegeben ist eine Liste von `Integer`-Werten. Gesucht ist die kleinste absolute Differenz zwischen zwei unterschiedlichen Elementen der Liste. Zusätzlich sollen die ursprünglichen Indexpositionen der beiden Elemente zurückgegeben werden.

Beispiel: In der Liste `[99, 1, 22, 33, 42, 55, 6, 77, 83, 100, 111, 122, 144]` beträgt die minimale Differenz `1`. Sie entsteht zwischen den Werten 99 und `100` an den Indexpositionen `0` und 9.

Die Brute-Force-Variante betrachtet alle möglichen Zahlenpaare, berechnet deren absolute Differenz und merkt sich das kleinste bisher gefundene Ergebnis.

```
record MinDifferenceResult(long difference, int index1, int index2) { }

static MinDifferenceResult findMinDifferenceBruteForce(List<Integer> values) {

    MinDifferenceResult best = new MinDifferenceResult(Long.MAX_VALUE, -1, -1);

    for (int i = 0; i < values.size(); i++) {
        for (int j = i + 1; j < values.size(); j++) {
            long difference = Math.abs((long) values.get(i) - values.get(j));

            if (difference < best.difference()) {
                best = new MinDifferenceResult(difference, i, j);
            }
        }
    }

    return best;
}
```

- Analysiere die Laufzeit der Brute-Force-Implementierung. Wie viele Zahlenpaare werden im schlimmsten Fall betrachtet?
- Implementiere eine effizientere Variante, indem du die Werte zunächst sortierst. Vergleiche anschließend nur noch benachbarte Werte.
- Gib auch bei der optimierten Variante die ursprünglichen Indexpositionen zurück. Wie kannst du diese Information beim Sortieren erhalten?
- Bestimme die Zeitkomplexität der optimierten Lösung. Berücksichtige dabei sowohl das Sortieren als auch den anschließenden Durchlauf.
- Vergleiche beide Varianten mit geeigneten Testdaten unterschiedlicher Grösse. Ab welcher Eingabegröße lohnt sich das Sortieren?

Hinweise

- Die Liste kann unsortiert sein und darf doppelte Werte enthalten.
- Es müssen zwei unterschiedliche Indexpositionen verwendet werden.
- Verwende den vorgegebenen `record MinDifferenceResult` für das Ergebnis.
- Die Differenz zweier `int`-Werte kann größer als `Integer.MAX_VALUE` sein. Deshalb wird vor der Subtraktion auf `long` erweitert.

Σ Prefixsummen

Aufgabe 1: Präfixprodukte berechnen

Schwierigkeit: leicht · Bearbeitungszeit: ca. 10–15 Min.

Übertrage das Prinzip der Präfixsummen auf die Multiplikation der jeweiligen Werte. Für jedes Arrayelement soll das Produkt aller Werte von Position 0 bis zur aktuellen Position berechnet werden.

Beispiel: `[2, 3, 4, 5, 6, 10] → [2, 6, 24, 120, 720, 7200]`.

```
static long[] prefixProducts(long[] values) {
    long[] result = new long[values.length];
    if (values.length == 0)
        return result;

    result[0] = values[0];
    for (int i = 1; i < values.length; i++) {
        // TODO: vorheriges Präfixprodukt fortschreiben
    }
    return result;
}
```

- Implementiere die Methode für `long[]`.
- Erstelle zusätzlich eine Variante für `List<Integer>`.
- Was passiert, wenn die Eingabedaten die Zahl 0 enthalten?
- Begründe die Laufzeitkomplexität.

Aufgabe 2: Temperatur-Monitoring

Schwierigkeit: mittel bis anspruchsvoll · Bearbeitungszeit: ca. 30 Min.

Eine Wetterstation zeichnet täglich die durchschnittliche Aussentemperatur auf. Du erhältst ein Array mit den gemessenen Temperaturen über einen längeren Zeitraum. Deine Aufgabe ist es, mithilfe von Präfixsummen verschiedene Temperaturauswertungen effizient durchzuführen.

```
int[] temperatures = {
        12, 15, 18, 14, 11, 9, 8, 13, 17, 20,
        22, 19, 16, 14, 12, 10, 7, 9, 13, 18,
        21, 24, 23, 19, 16, 14, 11, 8, 10, 15
};
```

Verwende einen Java-Record, um die Ergebnisse sauber zu kapseln.

Teilaufgabe 1: Präfixsummen berechnen

Implementiere eine Methode, die die Präfixsummen des Temperatur-Arrays berechnet.

```
Temperaturen: [12, 15, 18, 14, 11]
Präfixsummen: [12, 27, 45, 59, 70]
```

Teilaufgabe 2: Temperaturbereich abfragen

Implementiere eine Methode, die die Summe der Temperaturen zwischen zwei beliebigen Tagen inklusive beider Grenzen berechnet. Die Berechnung soll in O(1) erfolgen.

Teilaufgabe 3: Wärmste Woche finden

Finde den zusammenhängenden Zeitraum von sieben Tagen mit der höchsten Temperatursumme.

Teilaufgabe 4: Kälteste Woche finden

Finde analog den Zeitraum von sieben Tagen mit der niedrigsten Temperatursumme.

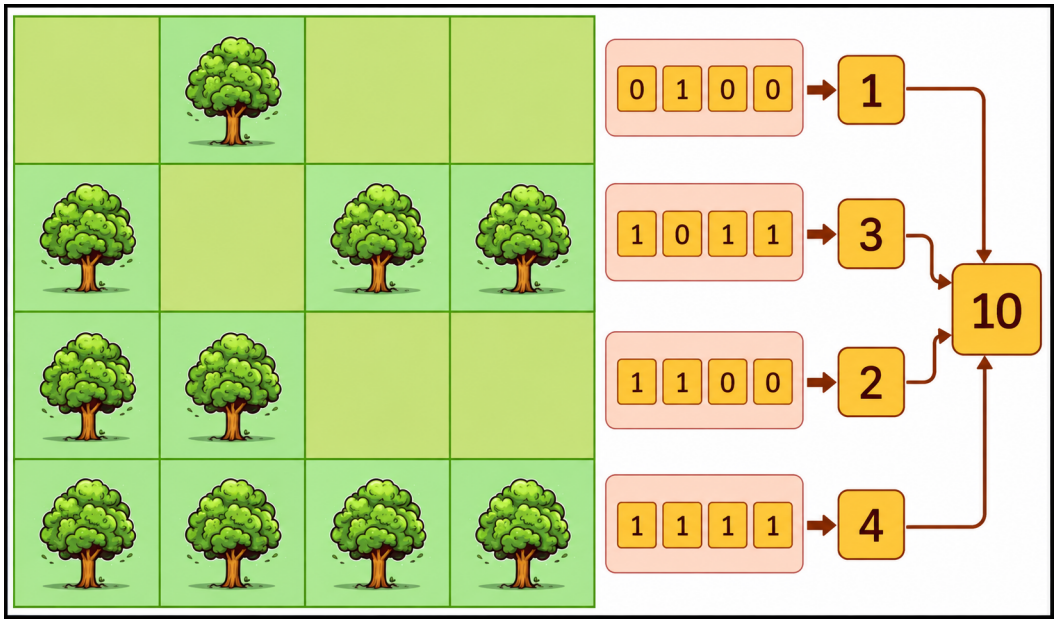
Kleine Bonus-Challenge

Finde den längsten zusammenhängenden Zeitraum, in dem die Temperatur an jedem einzelnen Tag über 15 °C lag.

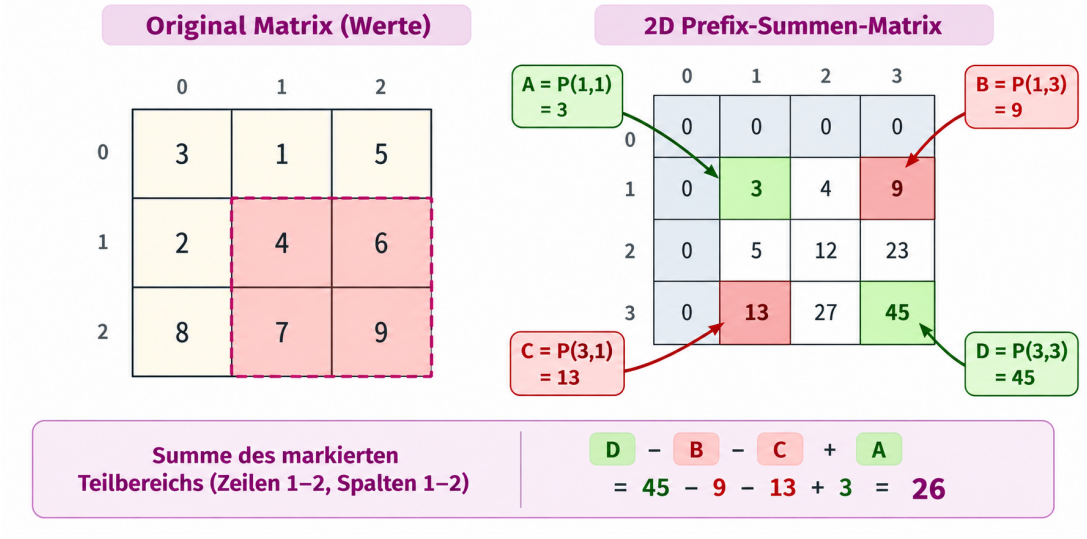
Aufgabe 3: 2D-Präfixsummen

Schwierigkeit: mittel bis anspruchsvoll · Bearbeitungszeit: ca. 30–40 Min.

Gegeben ist eine Matrix mit ganzzahligen Werten. Erzeuge eine Präfixsummenmatrix, mit der sich die Summe eines beliebigen rechteckigen Teilbereichs effizient bestimmen lässt.



Beispiel: Ein 4×4-Raster mit Bäumen. Die Zahl der Bäume pro Zeile ist rechts als Zahlenrepräsentation und Zeilensumme



2D-Präfixsummen: Aus der Präfixmatrix lässt sich die Summe des markierten Rechtecks mit vier Werten berechnen.

Verwende für eine erste Berechnung die folgende Matrix:

```
[[3, 1, 5],
 [2, 4, 6],
 [8, 7, 9]]
```

Für den Bereich von Zeile 1 bis 2 und Spalte 1 bis 2 beträgt die Summe 4 + 6 + 7 + 9 = 26.

```java
class PrefixSum2D {
    private final int[][] prefix;

    PrefixSum2D(int[][] matrix) {
        int rows = matrix.length;
        int cols = rows == 0 ? 0 : matrix[0].length;
        prefix = new int[rows + 1][cols + 1];
        // TODO: Präfixsummenmatrix aufbauen
    }

    // Koordinaten inklusive: (r1,c1) bis (r2,c2)
    long sumRegion(int r1, int c1, int r2, int c2) {
        // TODO: Rechtecksumme mit vier Präfixwerten berechnen
        return 0;
    }
}
```

- Implementiere den Aufbau der 2D-Präfixsummenmatrix.
- Implementiere `sumRegion(r1, c1, r2, c2)`.
- Teste Randbereiche, einzelne Zellen, die gesamte Matrix und negative Werte.
- Vergleiche Berechnungen mithilfe wiederholter direkter Bereichssummen mit dem Aufbau von Pärfixsummen plus anschließenden O(1)-Abfragen.

Formel Mit einer zusätzlichen Null-Zeile und Null-Spalte lautet die Rechteckformel: Pre[r2+1][c2+1] − Pre[r1][c2+1] − Pre[r2+1][c1] + Pre[r1][c1].

Kleine Bonus-Challenge

Gehen wir von einem größeren Datenbestand aus. Wir wollen für Planquadrate der Größe 5 x 5 folgende Dinge ermitteln:

- Finde das Rechteck mit dem höchsten Baumbestand.
- Finde das Rechteck mit dem niedrigsten Baumbestand.
- Gib die Koordinaten beider Rechtecke zurück.
- Bonus: Ermittle die durchschnittliche Baumdichte des besten Rechtecks.

Die bereitegstellte Matrix enthält bewusst:

- Einen dichten Waldabschnitt links oben.
- Einen besonders dichten Bereich in der Mitte rechts.
- Einen eher dünn bewachsenen Bereich unten.

Sliding Windows

Aufgabe 1: Maximale Summe eines Teilarrays fester Länge

Schwierigkeit: leicht · *Bearbeitungszeit: ca. 25 Min.*

Schreibe eine Methode `maxSum(int[] values, int k)`, die die maximale Summe von k aufeinanderfolgenden Elementen eines Arrays zurückgibt.

Teil A – Naiver Ansatz

Gegeben ist dieser Brute-Force-Ansatz: Untersuche alle möglichen zusammenhängenden Teilarrays der Länge k und berechne jeweils deren Summe.

```
static int maxSumNaive(int [] values, int k) {
    int best = Integer.MIN_VALUE;

    for (int start = 0; start <= values.length - k; start++) {
        int currentSum = 0;

        for (int i = start; i < start + k; i++) {
            currentSum += values[i];
        }

        if (currentSum > best) {
            best = currentSum;
        }
    }

    return best;
}
```

- Vollziehe die Brute-Force-Lösung nach und teste sie mit verschiedenen Eingaben.
- Bestimme die Laufzeitkomplexität in Abhängigkeit von n und k.
- Überlege, ob bei jedem Durchlauf wirklich alle Elemente erneut addiert werden müssen.
- Was haben zwei aufeinanderfolgende Fenster gemeinsam?

Teil B – Sliding Window

Implementiere eine effizientere Variante mithilfe der Sliding-Window-Technik.

- Berechne zunächst die Summe des ersten Fensters.
- Verschiebe das Fenster schrittweise um eine Position nach rechts.
- Ziehe das Element ab, das das Fenster verlässt.
- Addiere das neue Element, das in das Fenster aufgenommen wird.
- Falls nötig, aktualisiere die bisher größte Summe.
- Bestimme die Zeit- und Speicherkomplexität und vergleiche beide Varianten.

Aufgabe 2: Durchschnitt der Werte in einem Fenster

Schwierigkeit: leicht · *Bearbeitungszeit: ca. 25 Min.*

Schreibe eine Methode `findMaxAverage(int[] values, int k)`, die den maximalen Durchschnittswert von k aufeinanderfolgenden Elementen eines Arrays als `double` zurückgibt.

Teil A – Naiver Ansatz

Gegeben ist dieser Brute-Force-Ansatz: Untersuche alle möglichen Teilarrays der Länge k und berechne jeweils deren Durchschnitt.

```
static int findMaxAverageNaive(int [] values, int k) {
    double best = Double.NEGATIVE_INFINITY;

    for (int start = 0; start <= values.length - k; start++) {
        int currentSum = 0;

        for (int i = start; i < start + k; i++) {
            currentSum += values[i];
        }

        double average = (double) currentSum / k;
        if (average > best) {
            best = average;
        }
    }

    return best;
}
```

- Vollziehe die Brute-Force-Lösung nach und teste sie mit verschiedenen Eingaben.
- Bestimme die Laufzeitkomplexität in Abhängigkeit von n und k.
- Muss die Berechnung des Durchschnitts bei jedem Fenster vollständig wiederholt werden?
- Welche Beziehung besteht zwischen der maximalen Summe und dem maximalen Durchschnitt bei gleicher Fenstergröße?

Teil B – Sliding Window

Implementiere eine effizientere Variante mithilfe der Sliding-Window-Technik.

- Berechne die Summe des ersten Fensters.
- Verschiebe das Fenster schrittweise um eine Position.
- Aktualisiere die Summe, indem du das linke Element abziehst und das neue rechte Element addierst.
- Berechne den Durchschnitt für das jeweils aktuelle Fenster.
- Vergleiche die Laufzeit- und Speicherkomplexität beider Varianten.

Aufgabe 3: Angenehme Wetterperiode

Schwierigkeit: mittel · *Bearbeitungszeit: ca. 30 Min.*

Eine Wetterstation zeichnet täglich die durchschnittliche Aussentemperatur auf. Ein zusammenhängender Zeitraum gilt als angenehm, wenn die Temperatur an höchstens einem Tag unter oder auf nur 15 °C fällt. Deine Aufgabe ist es, den längsten zusammenhängenden Zeitraum zu finden, der diese Bedingung erfüllt.

Lernziel: Zwei Pointer, Fenster dynamisch vergrößern und verkleinern.

Beispieldaten

```
int[] temperatures = {16, 18, 12, 17, 19, 14, 20, 21, 13, 22, 24, 18};
```

Teil A – Verständnis

Vollziehe das Beispiel nach:

- Überlege, warum ein einfacher Zähler wie bei der längsten Wärmeperiode nicht mehr ausreicht.
- Welche Informationen musst du über das aktuelle Fenster speichern?
- Wann muss das Fenster von links verkleinert werden?

Teil B – Sliding Window

Implementiere eine Methode, die den längsten gültigen Zeitraum zurückgibt. Verwende zwei Pointer (`left` und `right`), um das Fenster zu definieren. Das Fenster darf höchstens einen Tag mit einer Temperatur von maximal 15 °C enthalten. Vergrößere das Fenster schrittweise nach rechts. Verkleinere das Fenster von links, sobald die Bedingung verletzt wird. Speichere die längste gültige Periode. Gib Starttag, Endtag (als Index) und Dauer zurück. Verwende dafür folgenden Record:

```
private record TemperatureStreak(int startDay, int endDay, int days) {}
```

Aufgabe 4: Produktionsabschnitt mit höchstens k Fehlern

Schwierigkeit: mittel · *Bearbeitungszeit: ca. 30 Min.*

Ein Qualitätskontrollsystem überwacht eine Produktionslinie. Jedes Array-Element beschreibt, ob ein produziertes Werkstück fehlerfrei oder fehlerhaft ist. Gesucht ist der längste zusammenhängende Produktionsabschnitt, der höchstens k fehlerhafte Werkstücke enthält.

Beispieldaten

```
// 1 = gut, 0 = fehlerhaft
int[] production = {1, 1, 0, 1, 1, 0, 1, 1, 1, 0, 1, 1};
```

Teil A – Verständnis

Vollziehe das Beispiel nach:

- Überlege, was passiert, wenn mehr als k Fehler im aktuellen Fenster auftreten?
- Warum muss das Fenster von links verkleinert werden?

Teil B – Sliding Window

Implementiere eine passende Methode. Zähle fehlerhafte Werkstücke im aktuellen Fenster. Vergrößere das Fenster schrittweise nach rechts. Sobald mehr als k Fehler enthalten sind, verkleinere das Fenster von links. Gib Start- und Endposition und längste gültige Produktionsperiode als Record zurück:

```
private record ProductionPeriod(int start, int end, int length) {}
```

Aufgabe 5: Längste Teilzeichenkette ohne doppelte Zeichen

Schwierigkeit: mittel/anspruchsvoll · Bearbeitungszeit: ca. 45 Min.

Schreibe `longestUniqueSubstring(String text)`, die eine längste zusammenhängende Teilzeichenkette ohne doppelte Zeichen zurückgibt.

Eingabe	Eine gültige Ausgabe
"abcabcbb"	"abc"
"bbbbb"	"b"
"abcdef"	"abcdef"

Teil A – Naiver Ansatz

Untersuche alle möglichen Teilzeichenketten und prüfe, ob sie doppelte Zeichen enthalten. Die folgende Variante nutzt `substring()` und eine separate Prüfung auf Eindeutigkeit.

```
static String longestUniqueSubstringNaive(String text) {
    String best = "";
    for (int start = 0; start < text.length(); start++) {
        for (int end = start + 1; end <= text.length(); end++) {
            String candidate = text.substring(start, end);
            if (containsOnlyUniqueChars(candidate) &&
                candidate.length() > best.length()) {
                best = candidate;
            }
        }
    }
    return best;
}

static boolean containsOnlyUniqueChars(String value) {
    Set<Character> seen = new HashSet<>();
    for (int i = 0; i < value.length(); i++) {
        if (!seen.add(value.charAt(i))) return false;
    }
    return true;
}
```

- Vollziehe die Brute-Force-Lösung nach und teste sie mit verschiedenen Zeichenketten.
- Bestimme die Laufzeitkomplexität.

Teil B – Sliding Window

Implementiere eine effizientere Variante mithilfe der Sliding-Window-Technik. Bestimme die Zeit- und Speicherkomplexität und vergleiche beide Varianten für kurze und sehr lange Strings und Fenster.

🧩 Rekursion und Memoization

Aufgabe 1: Fibonacci – von exponentiell zu linear

Schwierigkeit: mittel · Bearbeitungszeit: ca. 20–25 Min.

Implementiere die Berechnung der Fibonacci-Zahlen zunächst rekursiv ohne Zwischenspeicherung. Ergänze anschließend Memoization mit HashMap oder Array sowie eine iterative Tabulation für `List<Integer>`.

```
static long fibRecursive(int n) {
    if (n <= 1) return n;
    return fibRecursive(n - 1) + fibRecursive(n - 2);
}

static long fibMemo(int n, long[] memo) {
    if (n <= 1) return n;
    if (memo[n] != -1) return memo[n];
    return memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);
}

static long fibIterative(int n) {
    if (n <= 1) return n;
    long a = 0, b = 1;
    for (int i = 2; i <= n; i++) {
        long next = a + b;
        a = b;
        b = next;
    }
    return b;
}
```

- Vergleiche die Anzahl rekursiver Aufrufe und die Laufzeit.
- Ordne Zeit- und Speicherkomplexität jeder Variante zu.
- Erkläre, warum die gezeigte iterative Variante nur konstanten zusätzlichen Speicher benötigt.

Randfälle long kann Fibonacci-Zahlen nur bis zum Index 92 korrekt darstellen. Initialisiere ein Memo-Array mit -1, damit 0 als gültiger Fibonacci-Wert gespeichert werden kann.

Aufgabe 2: Anzahl möglicher Wege im Gitter

Schwierigkeit: mittel · Bearbeitungszeit: ca. 20–25 Min.

Eine Spielfigur steht links oben in einem rechteckigen Gitter und darf pro Zug nur einen Schritt nach rechts oder nach unten gehen. Berechne die Anzahl unterschiedlicher Wege zum rechten unteren Feld.

```
static long countPaths(int row, int col, int[][] memo) {
    if (row == 0 || col == 0)
        return 1;
    if (memo[row][col] != -1)
        return memo[row][col];
    // TODO: Wege von oben und links kombinieren
    return 0;
}
```

- Formuliere die rekursive Beziehung und Basisfälle.
- Ergänze Memoization und vergleiche die Laufzeit.
- Implementiere eine iterative Dynamic-Programming-Variante.
- **Bonus**: Bestimmte Felder sind blockiert. Erweitere die Lösung entsprechend.
  Tipp: Nutze ein zusätzliches `boolean[][]`.

Aufgabe 3: Edit Distance (Levenshtein-Distanz)

Schwierigkeit: anspruchsvoll · Bearbeitungszeit: ca. 45 Min.

Die Edit Distance zweier Strings ist die minimale Anzahl an Einfüge-, Lösch- und Ersetzoperationen, um den einen `String` in den anderen umzuwandeln.

String 1	String 2	Erwartete Distanz
kitten	sitting	3
haus	maus	1
	abc	3
abc	abc	0

```java
static int editDistance(String a, String b) {
    int[][] dp = new int[a.length() + 1][b.length() + 1];
    // TODO: Basisfälle für leere Präfixe initialisieren
    // TODO: Einfügen, Löschen und Ersetzen abwägen
    return dp[a.length()][b.length()];
}
```

- Definiere den Zustand anhand der Präfixlängen beider Strings.
- Formuliere die Basisfälle für leere Strings.
- Implementiere zunächst die rekursive Lösung und ergänze Memoization.
- Setze die Lösung iterativ um und bestimme Zeit- und Speicherkomplexität.

Hilfestellung

Die Edit Distance gibt an, wie viele Änderungen mindestens nötig sind, um ein Wort in ein anderes umzuwandeln. Dafür gibt es drei erlaubte Änderungen – jede zählt 1:

- Einfügen: Ein Zeichen hinzufügen.
- Löschen: Ein Zeichen entfernen.
- Ersetzen: Ein Zeichen austauschen.

Das lässt sich formal so schreiben:

$$D(i,j) = \begin{cases} j & \text{wenn } i = 0 \\ i & \text{wenn } j = 0 \\ D(i-1, j-1) & \text{wenn Zeichen gleich} \\ 1 + \min \begin{cases} D(i, j-1) \\ D(i-1, j) \\ D(i-1, j-1) \end{cases} & \text{wenn Zeichen verschieden} \end{cases}$$

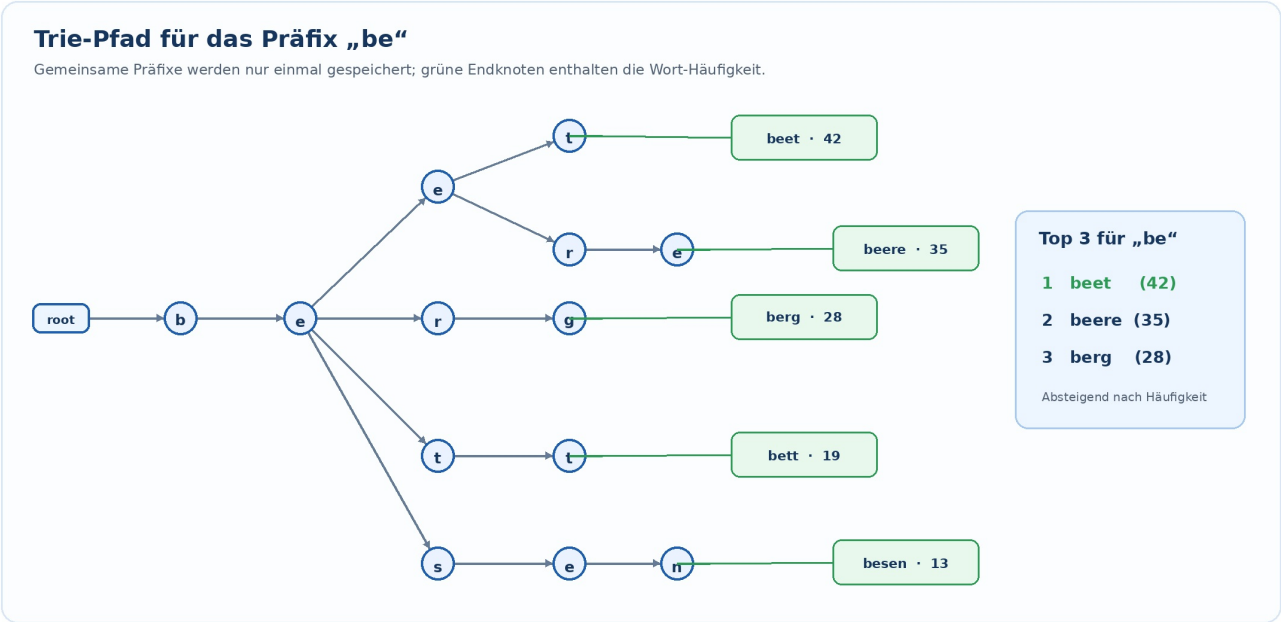
Möglichkeit	Bedeutung
`D(i, j-1)`	Zeichen einfügen
`D(i-1, j)`	Zeichen löschen
`D(i-1, j-1)`	Zeichen ersetzen

🌳 Tries und Autocomplete

Aufgabe 1: `Trie` verstehen und Autocomplete mit Ranking erweitern

Schwierigkeit: mittel · Bearbeitungszeit: ca. 30 Min.

Ein `Trie` speichert Wörter entlang gemeinsamer Präfixpfade. Die Grafik zeigt die Wörter „beet“, „beere“, „berg“, „bett“ und „besen“. Die Zahlen an den Wortenden geben die jeweilige Nutzungshäufigkeit an.



`Trie` mit gemeinsam genutzten Präfixen und Häufigkeitswerten an den vollständigen Wörtern.

Erweitere die vorhandene Autocomplete-Funktion so, dass für ein eingegebenes Präfix die k häufigsten passenden Wörter zurückgegeben werden. Verwende als Beispieldaten: `beet` (42), `beere` (35), `berg` (28), `bett` (19) und `besen` (13). Für das Präfix „be“ und k = 3 sollen „beet“, „beere“ und „berg“ in dieser Reihenfolge zurückgegeben werden.

Als Ausgangsbasis dient ein vereinfachter Ausschnitt aus der bereitgestellten `Trie`-Implementierung:

```
class TrieNode {
    final Map<Character, TrieNode> children = new HashMap<>();
    String word;
    int ranking;
}

class Trie {
    private final TrieNode root = new TrieNode();

    void insert(String word, int rankingOffset) {
        TrieNode current = root;
        for (char ch : word.toCharArray()) {
            current.children.putIfAbsent(ch, new TrieNode());
            current = current.children.get(ch);
        }
        current.word = word;
        current.ranking += rankingOffset;
    }

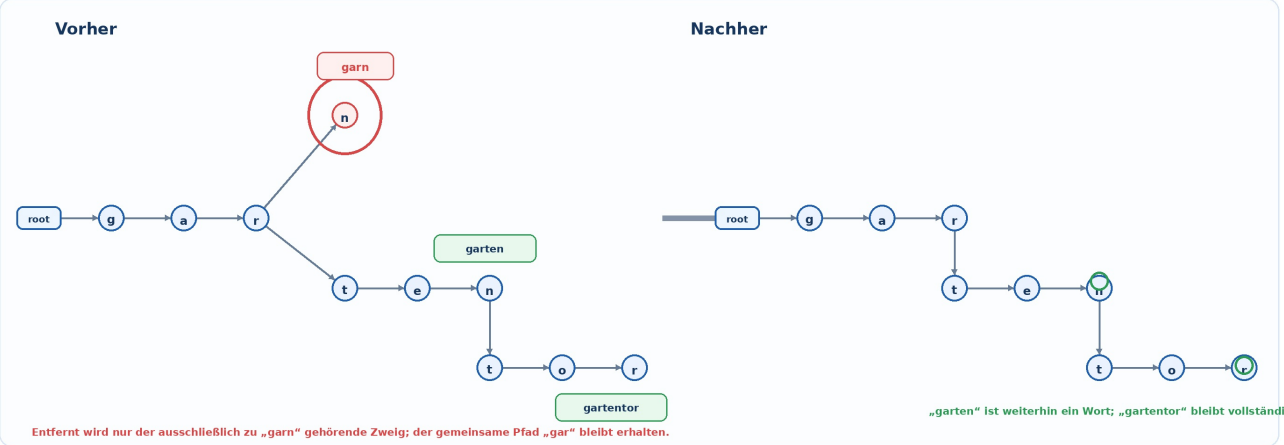
    // TODO: Präfixknoten finden und passende Wörter sammeln
    // TODO: Nach ranking absteigend sortieren und auf k begrenzen
}
```

- Lege fest, an welcher Stelle im `Trie` die Häufigkeit eines vollständigen Wortes gespeichert wird.
- Implementiere zunächst das Sammeln aller Treffer unterhalb des Präfixknotens und sortiere sie nach Häufigkeit.
- Diskutiere den Aufwand, wenn sehr viele Treffer zum Präfix passen.

Aufgabe 2: Wörter aus einem `Trie` löschen

Schwierigkeit: mittel bis anspruchsvoll · Bearbeitungszeit: ca. 30 Min.

Implementiere eine Methode `delete(String word)`. Sie soll ein Wort aus dem Trie entfernen, ohne andere gespeicherte Wörter zu beschädigen. Gemeinsam genutzte Präfixknoten müssen erhalten bleiben.



Beim Löschen von „garn“ entfällt nur dessen eigener Zweig; der gemeinsame Pfad „gar“ bleibt für „garten“ und „gartentor“ erhalten.

```
boolean delete(String word) {
    // TODO: Pfad zum Wortende finden
    // TODO: Wortende-Markierung entfernen
    // TODO: Von unten nach oben leere Knoten entfernen
    return false;
}
```

- Wenn ein Wort Präfix eines anderen Wortes ist, darf der gemeinsame Pfad nicht entfernt werden. Beim Löschen von „garten“ muss der Pfad erhalten bleiben, weil „gartentor“ weiterhin gespeichert ist.
- Entferne nicht mehr benötigte Knoten von unten nach oben bis zur ersten Verzweigung oder bis zu einem Knoten, der weiterhin ein Wortende markiert.
- Teste mit „garn“, „garten“, „gartentor“ und „gartentorschloss“: Lösche zuerst „garn“ und anschließend „garten“. Prüfe jeweils, dass die übrigen Wörter weiterhin gefunden werden. Lösche dann „gartentorschloss“: Es sollte nur noch „gartentor“ gespeichert sein.

Aufgabe 3: Longest Common Prefix

Schwierigkeit: leicht bis mittel · Bearbeitungszeit: ca. 15 Min.

Bestimme das längste gemeinsame Präfix aller in einem `Trie` gespeicherten Wörter. Überlege, wie du vom Wurzelknoten ausgehst und unter welchen Bedingungen du den Pfad fortsetzen kannst.

```
["stopping", "stopped", "stopwatch"]
["air", "airflow"]
```

🏁 Performance-Finale: messen, erklären, optimieren

Aufgabe 1: Engpässe in einer Tabellenoberfläche nachvollziehen

Schwierigkeit: mittel · Bearbeitungszeit: ca. 30 Min.

Eine Tabellenoberfläche funktioniert mit wenigen Einträgen flüssig, wird bei großen Datenmengen von 100.000 jedoch deutlich langsamer. Zusätzlich werden Icons dynamisch geladen. Analysiere die Anwendung systematisch, statt auf Verdacht zu optimieren.

```
// Illustrative Ausgangsskizze – an die konkrete UI anpassen
List<Row> rows = new LinkedList<>();
for (int i = 0; i < data.size(); i++) {
    Row row = buildRow(data.get(i));
    row.setIcon(loadIcon(row.iconUrl())); // möglicherweise I/O pro Zeile
    rows.add(row);
}

for (int i = 0; i < rows.size(); i++) {
    render(rows.get(i)); // Indexzugriff auf LinkedList
}
```

- Formuliere Hypothesen: Datenstruktur, Rendering, unnötige Objekterzeugung, I/O oder wiederholte Berechnungen.
- Reproduziere den Engpass mit einer definierten Datenmenge und miss das Ausgangsverhalten.
- Nutze `VisualVM`, um CPU-Hotspots und gegebenenfalls Speicherprobleme zu untersuchen.
- Identifiziere eine konkrete Ursache und formuliere eine gezielte Optimierung.
- Miss erneut unter möglichst vergleichbaren Bedingungen und dokumentiere die Veränderung.

Profiler-Tipp Achte auf Hotspots, Methodenaufrufe und die Anzahl der Aufrufe. Bei `LinkedList` ist wiederholter indexbasierter Zugriff teuer; dynamisches Laden pro Zeile kann I/O oder Wartezeiten verursachen. Belege die Ursache mit Messdaten.

Aufgabe 2: Optimierung begründen und kommunizieren

Schwierigkeit: mittel · Bearbeitungszeit: ca. 10 Min.

- Welches Symptom war sichtbar?
- Was war der nachgewiesene Engpass?
- Welche Änderung wurde vorgenommen und welcher Trade-off entsteht?
- Wie wurde die Verbesserung überprüft?

Optional: JMH als Vertiefung / Selbststudium

Schwierigkeit: anspruchsvoll · Bearbeitungszeit: ca. 45 Min.

Erstelle einen einfachen JMH-Benchmark für eine kleine, isolierte Operation. Vergleiche ihn mit deiner eigenen `nanoTime()`-Messung und erkläre die Vorteile von Warm-up, mehreren Messiterationen und kontrollierten Benchmark-Bedingungen.

```
import org.openjdk.jmh.annotations.*;
import java.util.stream.IntStream;

@State(Scope.Thread)
public class ExampleBenchmark {
    private int[] values;

    @Setup
    public void setup() {
        values = IntStream.range(0, 10_000).toArray();
    }

    @Benchmark
    public int sumValues() {
        int sum = 0;
        for (int value : values) sum += value;
        return sum;
    }
}
```